

Seller and Inventory Management System

Database Management System — Mini Project Report

Task Name: Task 3 | Subject: DBMS

Objective	Design and implement a relational database to manage sellers, their products, and live stock
Core Tables	Seller, Product, Inventory (Stock)
Key Features	Seller–product mapping, stock availability tracking, inventory status reporting

1. Introduction

E-commerce and marketplace platforms depend on accurate, real-time inventory data. Each seller on the platform lists one or more products, and each product carries stock information that must be tracked continuously so that customers only see items that are actually available for purchase. The **Seller and Inventory Management System** is a relational database designed to model this exact problem: it links sellers to the products they own, links products to their stock records, and produces reports that show, at any point in time, what is in stock, what is low on stock, and what has run out.

This report covers the five requirements of the task: creating the Seller and Inventory tables, establishing relationships between sellers, products and stock, maintaining seller product information, tracking available and unavailable products, and generating inventory status reports.

1.1 Objectives

- Design a normalized schema that avoids data redundancy between seller, product and stock data.
- Enforce referential integrity using primary and foreign keys.
- Allow each seller to manage multiple products, and each product to have exactly one live stock record.
- Classify products as Available, Low Stock, or Out of Stock automatically from quantity data.
- Provide SQL queries and views that generate ready-to-use inventory status reports.

2. Entity–Relationship Overview

The system is built around three core entities — **Seller**, **Product**, and **Inventory** — plus a supporting **Category** entity used to group products. The relationships between them are as follows:

Relationship	Cardinality	Description
Seller → Product	One-to-Many	One seller can list many products; every product belongs to exactly one seller (se
Category → Product	One-to-Many	One category (e.g. Electronics, Grocery) can classify many products.
Product → Inventory	One-to-One	Each product has exactly one live stock record holding quantity and status.
Seller → Inventory	One-to-Many (derived)	Reachable through Product; used to report a seller's total stock position.

Table 2.1 – Relationships between entities

In ER-diagram terms: SELLER (1)—<places>—(M) PRODUCT (1)—<has>—(1) INVENTORY, with CATEGORY (1)—<classifies>—(M) PRODUCT feeding in from the side. This keeps seller information, catalogue information, and stock information in three separate tables so that updating a stock quantity never requires touching seller or product data, and vice versa (this follows Third Normal Form / 3NF).

3. Table Creation — Schema Design

Below are the four tables that make up the schema, along with their column definitions, data types and constraints.

3.1 Seller Table

Column	Data Type	Constraint
seller_id	INT	PRIMARY KEY, AUTO_INCREMENT
seller_name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
phone_number	VARCHAR(15)	NOT NULL
address	VARCHAR(255)	NULL
city	VARCHAR(50)	NULL
registration_date	DATE	DEFAULT CURRENT_DATE
rating	DECIMAL(2,1)	DEFAULT 0.0, CHECK (rating BETWEEN 0 AND 5)

Table 3.1 – Seller

3.2 Category Table

Column	Data Type	Constraint
category_id	INT	PRIMARY KEY, AUTO_INCREMENT
category_name	VARCHAR(50)	NOT NULL, UNIQUE

Table 3.2 – Category

3.3 Product Table

Column	Data Type	Constraint
product_id	INT	PRIMARY KEY, AUTO_INCREMENT
product_name	VARCHAR(150)	NOT NULL
category_id	INT	FOREIGN KEY → Category(category_id)
seller_id	INT	FOREIGN KEY → Seller(seller_id), NOT NULL
unit_price	DECIMAL(10,2)	NOT NULL, CHECK (unit_price >= 0)
description	VARCHAR(255)	NULL
date_added	DATE	DEFAULT CURRENT_DATE

Table 3.3 – Product

3.4 Inventory (Stock) Table

Column	Data Type	Constraint
stock_id	INT	PRIMARY KEY, AUTO_INCREMENT
product_id	INT	FOREIGN KEY → Product(product_id), UNIQUE, NOT NULL
quantity_available	INT	NOT NULL, DEFAULT 0, CHECK (quantity_available >= 0)
reorder_level	INT	DEFAULT 10
quantity_sold	INT	DEFAULT 0
status	VARCHAR(20)	DEFAULT 'Available' — Available / Low Stock / Out of Stock
last_updated	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

Table 3.4 – Inventory / Stock

The one-to-one link between Product and Inventory is enforced by making **product_id** both a foreign key and a UNIQUE column in the Inventory table — this guarantees a product cannot have more than one active stock record.

4. SQL – Table Creation Scripts

```
CREATE TABLE Seller (
    seller_id      INT AUTO_INCREMENT PRIMARY KEY,
    seller_name    VARCHAR(100) NOT NULL,
    email          VARCHAR(100) UNIQUE NOT NULL,
    phone_number   VARCHAR(15) NOT NULL,
    address        VARCHAR(255),
    city           VARCHAR(50),
    registration_date DATE DEFAULT (CURRENT_DATE),
    rating         DECIMAL(2,1) DEFAULT 0.0
                CHECK (rating BETWEEN 0 AND 5)
);

CREATE TABLE Category (
    category_id    INT AUTO_INCREMENT PRIMARY KEY,
    category_name  VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE Product (
    product_id     INT AUTO_INCREMENT PRIMARY KEY,
    product_name   VARCHAR(150) NOT NULL,
    category_id    INT,
    seller_id      INT NOT NULL,
    unit_price     DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),
    description     VARCHAR(255),
    date_added     DATE DEFAULT (CURRENT_DATE),
    FOREIGN KEY (category_id) REFERENCES Category(category_id),
    FOREIGN KEY (seller_id) REFERENCES Seller(seller_id)
        ON DELETE CASCADE
);

CREATE TABLE Inventory (
    stock_id       INT AUTO_INCREMENT PRIMARY KEY,
    product_id     INT NOT NULL UNIQUE,
    quantity_available INT NOT NULL DEFAULT 0
                CHECK (quantity_available >= 0),
    reorder_level  INT DEFAULT 10,
    quantity_sold  INT DEFAULT 0,
    status         VARCHAR(20) DEFAULT 'Available',
    last_updated   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
        ON DELETE CASCADE
);
```

A trigger (or application-layer logic) keeps the *status* column consistent with the quantity whenever stock is updated:

```
DELIMITER //
CREATE TRIGGER trg_update_status
BEFORE UPDATE ON Inventory
FOR EACH ROW
BEGIN
    IF NEW.quantity_available = 0 THEN
        SET NEW.status = 'Out of Stock';
    ELSEIF NEW.quantity_available <= NEW.reorder_level THEN
        SET NEW.status = 'Low Stock';
    ELSE
        SET NEW.status = 'Available';
    END IF;
END IF;
```

```
END //  
DELIMITER ;
```

5. Maintaining Seller Product Information

Because every row in Product carries a `seller_id`, the full catalogue belonging to a seller — and the seller's own contact details — can be retrieved with a single join. This satisfies requirement 3: keeping seller product information accessible and up to date.

```
-- All products listed by a specific seller, with seller contact info
SELECT  s.seller_id, s.seller_name, s.email, s.phone_number,
        p.product_id, p.product_name, p.unit_price, c.category_name
FROM    Seller s
JOIN    Product p  ON s.seller_id = p.seller_id
LEFT JOIN Category c ON p.category_id = c.category_id
WHERE   s.seller_id = 101
ORDER BY p.product_name;
```

Adding a new product for a seller and its opening stock record in one transaction:

```
START TRANSACTION;

INSERT INTO Product (product_name, category_id, seller_id, unit_price, description)
VALUES ('Wireless Mouse', 3, 101, 799.00, 'Ergonomic 2.4GHz wireless mouse');

INSERT INTO Inventory (product_id, quantity_available, reorder_level)
VALUES (LAST_INSERT_ID(), 50, 10);

COMMIT;
```

Updating a seller's product price (only if they own the product):

```
UPDATE Product
SET    unit_price = 749.00
WHERE  product_id = 2045 AND seller_id = 101;
```

6. Tracking Available and Unavailable Products

Requirement 4 is handled by the *status* column on Inventory (kept accurate by the trigger above) and by queries that filter on it or on the raw quantity.

```
-- Products currently available for sale
SELECT p.product_id, p.product_name, i.quantity_available, i.status
FROM   Product p
JOIN   Inventory i ON p.product_id = i.product_id
WHERE  i.status = 'Available'
ORDER BY p.product_name;

-- Products that are unavailable (out of stock)
SELECT p.product_id, p.product_name, s.seller_name, i.status, i.last_updated
FROM   Product p
JOIN   Inventory i ON p.product_id = i.product_id
JOIN   Seller s   ON p.seller_id = s.seller_id
WHERE  i.status = 'Out of Stock'
ORDER BY i.last_updated DESC;

-- Products approaching reorder level (early warning)
SELECT p.product_name, i.quantity_available, i.reorder_level
FROM   Product p
JOIN   Inventory i ON p.product_id = i.product_id
WHERE  i.quantity_available <= i.reorder_level
      AND i.quantity_available > 0;
```

7. Generating Inventory Status Reports

Requirement 5 asks for consolidated inventory reports. These are best exposed as SQL views, so any reporting tool or application page can query them directly without repeating the join logic.

7.1 Full inventory status report

```
CREATE VIEW vw_inventory_report AS
SELECT  s.seller_name,
        p.product_name,
        c.category_name,
        p.unit_price,
        i.quantity_available,
        i.quantity_sold,
        i.status,
        i.last_updated
FROM    Inventory i
JOIN    Product  p ON i.product_id = p.product_id
JOIN    Seller   s ON p.seller_id  = s.seller_id
LEFT JOIN Category c ON p.category_id = c.category_id;

-- Usage
SELECT * FROM vw_inventory_report ORDER BY seller_name, product_name;
```

7.2 Stock summary by seller

```
SELECT  s.seller_name,
        COUNT(p.product_id)                AS total_products,
        SUM(CASE WHEN i.status='Available'  THEN 1 ELSE 0 END) AS available_count,
        SUM(CASE WHEN i.status='Low Stock'   THEN 1 ELSE 0 END) AS low_stock_count,
        SUM(CASE WHEN i.status='Out of Stock' THEN 1 ELSE 0 END) AS out_of_stock_count,
        SUM(i.quantity_available)           AS total_units_in_stock
FROM    Seller s
JOIN    Product p  ON s.seller_id = p.seller_id
JOIN    Inventory i ON p.product_id = i.product_id
GROUP BY s.seller_id, s.seller_name
ORDER BY total_units_in_stock DESC;
```

7.3 Overall stock-status distribution

```
SELECT status, COUNT(*) AS product_count
FROM    Inventory
GROUP BY status;
```

Sample output of the status-distribution report:

Status	Product Count
Available	142
Low Stock	27
Out of Stock	9

Table 7.1 – Example inventory status distribution

8. Conclusion

The Seller and Inventory Management System schema separates seller identity, product catalogue, and stock levels into three linked tables, connected through clean one-to-many and one-to-one relationships. Primary and foreign keys enforce referential integrity, a status-updating trigger keeps availability accurate automatically, and a small set of joins and views cover every requirement of the task: table creation, relationship modelling, seller-product maintenance, availability tracking, and inventory reporting. The design is normalized to 3NF, which keeps updates isolated — changing a stock count never risks corrupting seller or catalogue data — and the schema can be extended later with tables for Orders, Warehouses, or Suppliers without restructuring what already exists.